SRS - AUA Rentals Vehicle Rental System v2.0

Oct 31, 2026 · @Ashar

Revision history and change log

Version 2.0 records three changes to `AUA Rentals OOP.cpp`: CH-02 and CH-03 are done for normal use, and CH-01 covers the View Rented screen only.

Version	Date	Summary
1.0	25 Oct 2026	First SRS, reverse-engineered from the fixed `car_rental.cpp`.
2.0	31 Oct 2026	Re-baselined on `AUA Rentals OOP.cpp`; adds CH-01 to CH-03 and a status column for every requirement.

ID	Change: before, then after	Code location	Requirements	Status in v2.0
CH-01	**Customer saw another customer's rented vehicle.** Before: rented-vehicle screens scanned other customers' rentals (v1.0 defect 7). After: View Rented Vehicles reads only the logged-in customer's row, `cusRentedVehicle[foundIndex][m][n]`.	`Rent::rentDetails`, menu option 2, line 764	FR-RNT-05, UR-03	Partly done. Return Vehicle (line 797) still loops over every customer.
CH-02	**A rented car could be rented by a second customer.** Before: an already-rented vehicle number could be accepted. After: the rent flow looks the vehicle up by identification number, checks `isVehAvailable`, and for a rented one prints VEHICLE ALREADY RENTED and returns to the rent menu. A customer already holding a vehicle is refused earlier, on `isCusAvailable` (line 651).	`Rent::rentDetails`, menu option 1, lines 651 and 704 to 712	FR-RNT-03, FR-RNT-04, NFR-03	Done for normal use. Depends on recursive menu returns (limitation L-01).

ID	Change: before, then after	Code location	Requirements	Status in v2.0
CH-03	**Letters at a menu prompt caused an infinite loop.** Before: `cin>>option` failed, stayed in a fail state and re-read the same bad input forever. After: every menu prompt tests `if(!(cin>>option))`, then calls `cin.clear()` and `cin.ignore(1000, '\n')`, prints INVALID INPUT and redisplays the menu.	Nine menu prompts: lines 285, 392, 495, 639, 885, 939, 1007, 1068, 1219	FR-INP-01, FR-INP-02, NFR-02	Done for menu prompts. Not yet for mileage, rent per km, return mileage, Y/N prompts or closed input (L-02).

1. Introduction

AUA Rentals is a C++ console program in which owners list vehicles, customers rent them by the kilometre, and an admin oversees users, vehicles and revenue; this SRS states what the current code does and how far it meets each requirement.

1.1 Purpose

To give developers, testers and the course instructor one agreed description of the system, so the code in `AUA Rentals OOP.cpp` can be checked against it requirement by requirement.

1.2 Scope

A single-machine, text-menu program. Owners register and list up to 3 vehicles each. Customers register, rent one vehicle at a time and return it for a per-kilometre charge. The admin views customers, owners, vehicles and total revenue. Data lives in plain text files. Online payment, GPS tracking, booking by date and a graphical interface are out of scope.

1.3 Definitions

Term	Meaning
Customer	A registered person who rents a vehicle
Owner	A registered person who lists vehicles for rent
Admin	The single built-in manager account

Term	Meaning
Mileage	The vehicle's odometer reading in km
Rent per km	The charge in RS for each km driven during a rental
Rental cost	(return mileage - mileage at pickup) x rent per km
Revenue	Sum of all rental costs collected since the data files were created
Met / Partly met / Not met	How far `AUA Rentals OOP.cpp` meets a requirement today (section 4)

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- Source file `AUA Rentals OOP.cpp` (1,528 lines, C++11, standard library only)
- SRS - AUA Rentals Vehicle Rental System, version 1.0, 25 Oct 2026

2. Overall description

The product is one standalone program with three portals behind a main menu, all sharing one in-memory data set that is saved to text files.

2.1 Product perspective

No network, database or third-party library. At start-up `loadAllData()` reads five text files; `saveAllData()` writes seven after every change. Global arrays hold the data, four classes hold the behaviour (`loginAccess`, `Register`, `Management`, `Rent`), and free functions draw the menus, which call one another recursively.

2.2 User classes

User	Entry point	Main goals
Customer	Customer portal	Register, log in, rent one vehicle, view it, return it and see the cost
Owner	Owner portal	Register, log in, add up to 3 vehicles, view and remove them

User	Entry point	Main goals
Admin	Admin portal (`admin` / `admin123`)	Review customers, owners, vehicles (all, available, rented) and total revenue

2.3 Operating environment

Any OS with a C++11 compiler, a console, and write access to the folder the program runs from.

2.4 Design and implementation constraints

- Fixed global arrays: 10 customers, 10 owners (`MAX_USERS`) and 3 vehicles per owner (the 3 is written directly into the code).
- Text-file persistence with `|` as the field separator; the code does not yet reject `|` in typed text.
- Console only; numbered menus; the screen is "cleared" by printing 25 blank lines.

2.5 Assumptions and dependencies

- One program instance uses the data files at a time.
- Customers are trusted to enter the true odometer reading on return.
- Currency is RS (rupees).

3. Requirements engineering

The v1.0 requirement set is kept, and version 2.0 adds a check of each requirement against the current code, so the document shows what the program does, not what it should do.

3.1 Process followed

Stage	What was done in v2.0
Elicitation	The developer reported three fixes (CH-01 to CH-03); every menu, prompt and message in the source was read again.

Stage	What was done in v2.0
Analysis	Each fix was mapped to the requirement it serves, and each requirement was traced to the lines that implement it.
Specification	IDs (UR, FR, NFR) and priorities are unchanged from v1.0; a status column was added.
Validation	Code inspection of each changed line and its neighbours; results are in section 7. Scripted runs are listed as future work.
Management	IDs are stable, so a change request can cite them; the tables in section 4 are the traceability matrix.

3.2 Stakeholders

Stakeholder	Interest
Customer	Easy registration, clear price, quick rent and return, no view of other customers' rentals
Owner	Control over which vehicles are listed, correct payment for km driven
Admin	A complete, accurate view of users, fleet and revenue
Developer and instructor	A system that is testable, readable and traceable to requirements

4. Requirements and status in v2.0

Of the 38 functional requirements, 22 are met, 10 are partly met and 6 are not met by `AUA Rentals OOP.cpp`; the status column is new in version 2.0.

Met: works as specified, by the scripted run or by inspecting the lines. **Partly met**: the main case works but a stated condition fails. **Not met**: no code for it yet.

Group	Total	Met	Partly met	Not met
User requirements (UR)	11	8	3	0
Functional requirements (FR)	38	22	10	6
Non-functional requirements (NFR)	9	5	4	0
All requirements	58	35	17	6

4.1 User requirements

ID	As a … I want to …	Priority	Status
UR-01	customer, register and log in with my email and password	Must	Met
UR-02	customer, see which vehicles are free and their rate per km	Must	Met
UR-03	customer, rent a free vehicle and later return it and see my cost	Must	Met
UR-04	owner, register and log in	Must	Met
UR-05	owner, add, view and remove my vehicles	Must	Partly met
UR-06	admin, log in to a protected portal	Must	Met
UR-07	admin, list and search customers and owners	Should	Partly met
UR-08	admin, see all, available and rented vehicles	Should	Met
UR-09	admin, see total revenue	Should	Met
UR-10	any user, keep my data after the program closes	Must	Met
UR-11	any user, get a clear message when I enter bad input	Should	Partly met

4.2 Registration and login

ID	Requirement	Priority	Status	Evidence and note
FRREG-01	Register a customer or an owner from name, phone, email and password with the next sequential ID.	Must	Met	`Register::registerCus`, `registerOwn`
FRREG-02	Reject a registration whose email already exists in the same user class.	Must	Met	Also rejects a repeated name (v1.0 defect 14, open).
FRREG-03	Reject empty fields, fields containing `|`, malformed emails and phones not 7 to 15 digits, + or -.	Should	Not met	No field validation in the register functions.
FRREG-04	Store at most 10 customers and 10 owners and say so at the limit.	Must	Met	`MAX_USERS` check at the top of each register function.
FRREG-05	Use the lowercased email as the username.	Must	Partly met	Email is the username but is not lowercased, so login is case-sensitive.
FRAUT-01	Grant customer access only when email and password match; otherwise print ACCESS DENIED.	Must	Met	`loginAccess::verifyCustomer`
FRAUT-02	Do the same for owners.	Must	Met	`loginAccess::verifyOwner`
FRAUT-03	Grant admin access only for the built-in admin credentials.	Must	Met	`loginAccess::verifyAdmin`

4.3 Owner and vehicle management

ID	Requirement	Priority	Status	Evidence and note
FR-OWN01	An owner adds a vehicle with name, identification number, mileage and rent per km; the ID is owner number plus sequence, such as 1-2.	Must	Met	`Register::registerVehicle`; IDs 1-1 and 1-2 seen in the run.
FR-OWN02	At most 3 vehicles per owner.	Must	Partly met	Limit works, but the message branch opens the customer portal with an owner index (line 229, v1.0 defect 5).
FR-OWN03	Vehicle identification numbers are unique across all owners; re-ask on a duplicate.	Must	Not met	The duplicate loop compares the wrong arrays (lines 242 to 253, defect 4).
FR-OWN04	Accept mileage of 0 or more and rent per km above 0 only.	Must	Not met	Values read with `cin>>` into `float`, unchecked.
FR-OWN05	An owner views profile, vehicle count and each vehicle with its status.	Must	Met	`ownPortal`, option 1
FR-OWN06	An owner removes a vehicle after a Y/N confirmation.	Must	Partly met	Confirmation works; an unknown number carries on with index -1 (line 1141, defect 10).
FR-OWN07	Refuse to remove a rented vehicle.	Must	Not met	No rented check before deletion (defect 11).
FR-OWN08	After a removal, renumber the remaining vehicle IDs and keep their rentals attached.	Should	Not met	IDs and `cusRentedVehicle` are not shifted (defect 11).

4.4 Renting and returning

ID	Requirement	Priority	Status	Evidence and note
FR-RNT01	A logged-in customer sees their profile and whether they hold a vehicle.	Must	Met	`cusPortal`
FR-RNT02	List only vehicles that are not rented, with owner, ID, name, number, mileage and rate.	Must	Partly met	Lists unrented vehicles with all fields except the owner's name.
FR-RNT03	Rent a vehicle by identification number after a Y/N confirmation; a rented or unknown number is refused.	Must	Met	**CH-02.** Run: Bilal typing `ABC-1` while Ali held it got VEHICLE ALREADY RENTED.
FR-RNT04	A customer holds at most one rented vehicle at a time.	Must	Met	`isCusAvailable` check at line 651.
FR-RNT05	A customer views only their own rented vehicle.	Must	Partly met	**CH-01.** View Rented is fixed (run: Bilal saw only his own). Return Vehicle (line 797) still lists every customer's rentals (run: Bilal was offered Ali's `ABC-1`).
FR-RNT06	On return the customer enters the odometer reading; a reading below the stored mileage is refused.	Must	Met	Lines 815 to 819.
FR-RNT07	Cost = (new mileage - old mileage) x rent per km; show it, add it to revenue, store the new mileage, free vehicle and customer.	Must	Met	Lines 821 to 830.

ID	Requirement	Priority	Status	Evidence and note
FR-RNT08	A customer can cancel a return and keep the vehicle rented.	Should	Met	Run: answering n printed RETURN VEHICLE CANCELLED.

4.5 Administration

ID	Requirement	Priority	Status	Evidence and note
FRADM-01	List all customers with ID, name, phone, email and rental status.	Should	Met	`Management::cusManagement`, option 1
FRADM02	Register a customer or an owner from the admin portal.	Could	Met	Options 2 of both management menus.
FRADM-03	Search a customer or an owner by email.	Should	Partly met	A found record prints, but NOT FOUND is printed after the menu returns (lines 360 to 365, defect 9).
FRADM04	List all owners with their vehicle counts.	Should	Partly met	Lists owners; the vehicle count is not shown.
FRADM-05	List all, only available, or only rented vehicles.	Should	Met	`Management::vehManagement`; the rented list prints RENTED correctly.
FRADM-06	Show total revenue in RS.	Should	Met	`Management::transManagement`

4.6 Persistence and input handling

ID	Requirement	Priority	Status	Evidence and note
FR-PER01	Load customers, owners, vehicles, revenue and active rentals from text files at start-up.	Must	Met	`loadAllData`
FR-PER02	Save after every change and on exit.	Must	Met	`saveAllData` after each register, rent, return, delete and exit.
FR-PER03	After a restart, rented vehicles and their customers still show as rented.	Must	Met	`RENT_DETAILS.txt` is read back at start-up.
FR-PER04	Write report files of available and rented vehicles.	Could	Met	`AVAILABLE_VEH_DATA.txt`, `RENTED_VEH_DATA.txt`
FR-PER05	Skip malformed lines in data files instead of crashing.	Should	Partly met	Incomplete lines stop the read; `stoi` and `stof` have no try/catch, so a bad number would throw.
FR-INP01	Reject a non-numeric menu or number entry and ask again.	Should	Partly met	**CH-03.** Nine menu prompts guarded (run: `abc` gave one INVALID INPUT and a redisplay). Mileage, rate, return mileage and Y/N prompts are unguarded.
FR-INP02	Show a message for a choice outside the list and redisplay the menu.	Should	Partly met	Message shown, but the rent menu sends an invalid choice to the customer portal (line 852) and the owner portal to the owner menu (line 1200).

ID	Requirement	Priority	Status	Evidence and note
FR-INP03	Back and Exit return to the previous menu; the program ends only from main menu option 4, and saves and ends if input is closed.	Must	Not met	Menus call each other. Run: after PROGRAM ENDED an earlier Return prompt reappeared, and at end of input the program crashed (exit code 139).

4.7 Non-functional requirements

ID	Category	Requirement	Status	Evidence and note
NFR-01	Usability	Every screen is a numbered menu with a prompt; messages are plain capitals text.	Met	All menus.
NFR-02	Reliability	Bad input never crashes the program or loops forever.	Partly met	Fixed for menu prompts (CH-03); unguarded numeric prompts and end of input still fail (L-02).
NFR-03	Data integrity	A vehicle is never rented to two customers and a customer never holds two vehicles.	Partly met	Enforced at the prompt (CH-02, FR-RNT04). Stale menu frames can resume after Exit (L-01), so "never" is not yet proven.
NFR-04	Persistence	Data survives a normal exit and a closed input stream.	Met	Data is saved after every change, so the crash at end of input loses nothing.
NFR-05	Performance	Every action responds instantly for 10 customers, 10 owners and 30 vehicles.	Met	Loops over at most 10 x 10 x 3 entries.

ID	Category	Requirement	Status	Evidence and note
NFR-06	Portability	Builds with any C++11 compiler using only the standard library.	Met	Compiled with `g++ -std=c++11 -Wall -Wextra`, no warnings.
NFR-07	Maintainability	Four classes plus helper functions; menus use loops, not recursion.	Partly met	Four classes exist; every menu is recursive.
NFR-08	Security (basic)	Portals require credentials; passwords are stored as plain text.	Met	See limitations, section 8.
NFR-09	Capacity	10 customers, 10 owners, 3 vehicles per owner, set by named constants.	Partly met	`MAX_USERS` exists; the 3 vehicles per owner is a literal in the code.

5. Interfaces, data files and business rules

The program talks to the user through the console and to disk through seven text files in its working folder.

5.1 External interfaces

- **User interface:** keyboard input and text output in a console; menu choices are read with `cin>>` and names, emails, passwords and vehicle numbers with `getline`.
- **File interface:** plain text files read at start-up and rewritten after every change.
- **Hardware and network:** none.

5.2 Data files

File	Content, one record per line (fields split by |)	Read back
`CUSTOMERS_DATA.txt`	id, name, phone, email, username, password, renting flag	Yes
`OWNERS_DATA.txt`	id, name, phone, email, username, password	Yes

File	Content, one record per line (fields split by |)	Read back
`VEHICLES_DATA.txt`	owner index, vehicle ID, name, number, mileage, rate per km, rented flag	Yes
`RENT_DETAILS.txt`	customer index, owner index, vehicle index, then a readable copy of customer, owner and vehicle details	Yes
`REVENUE.txt`	total revenue	Yes
`AVAILABLE_VEH_DATA.txt`, `RENTED_VEH_DATA.txt`	report files for humans	No

Naming note: the flags `isVehAvailable` and `isCusAvailable` are true when a vehicle is rented and when a customer holds one, the opposite of what the names suggest (v1.0 defect 15, open).

5.3 Business rules

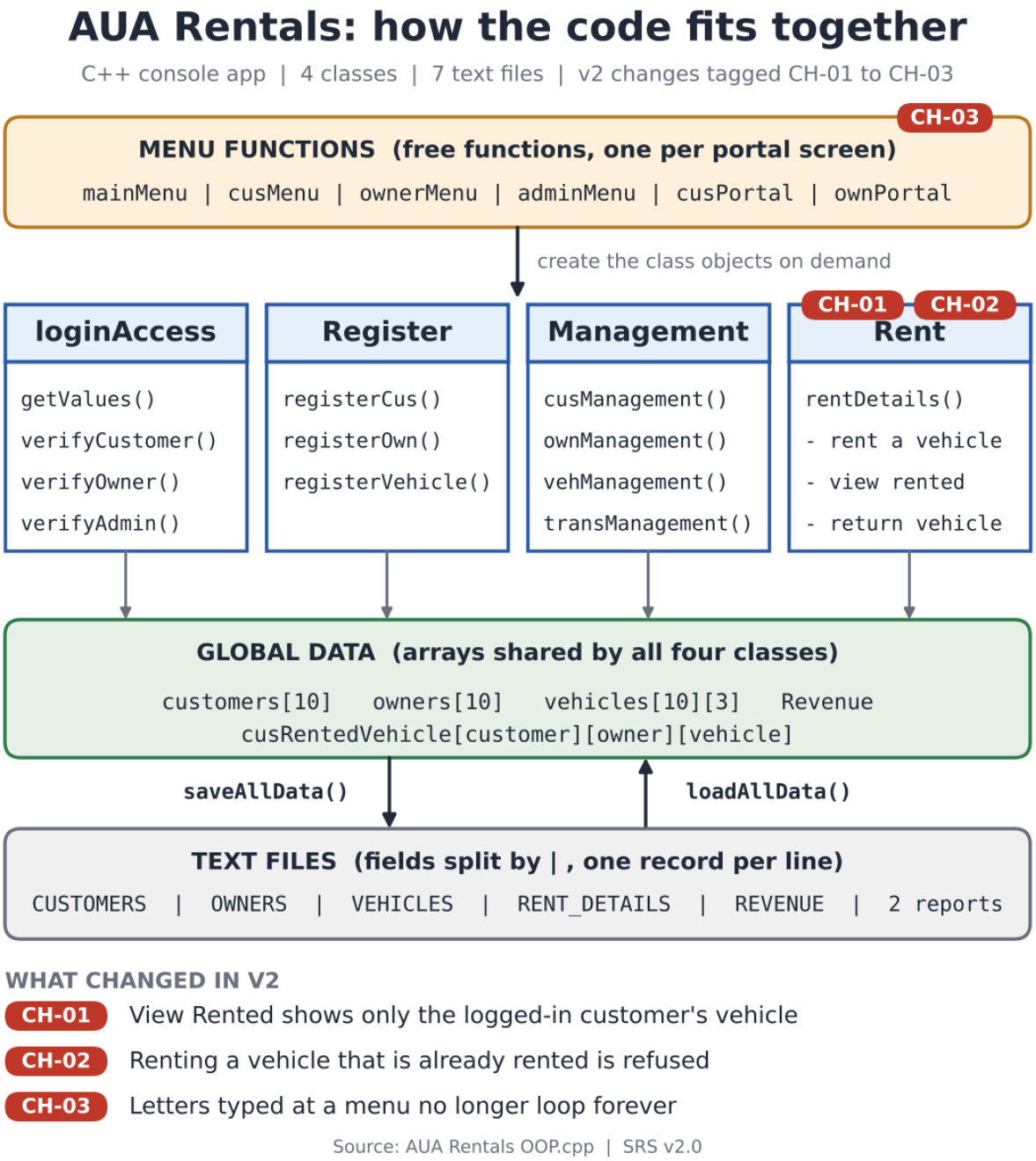
- **BR-01:** Rental cost = (return mileage - pickup mileage) x rent per km, so no cost accrues without driven distance.
- **BR-02:** All rental cost is credited to total revenue and paid to the owner at return.
- **BR-03:** The return mileage becomes the vehicle's new pickup mileage.
- **BR-04 (new in v2.0):** A rented vehicle cannot be rented again until it is returned (CH-02).

6. UML models

Three diagrams are new or redrawn for version 2.0; the use case diagram (v1.0 section 6.1) and domain model (v1.0 section 6.3) are unchanged and still valid.

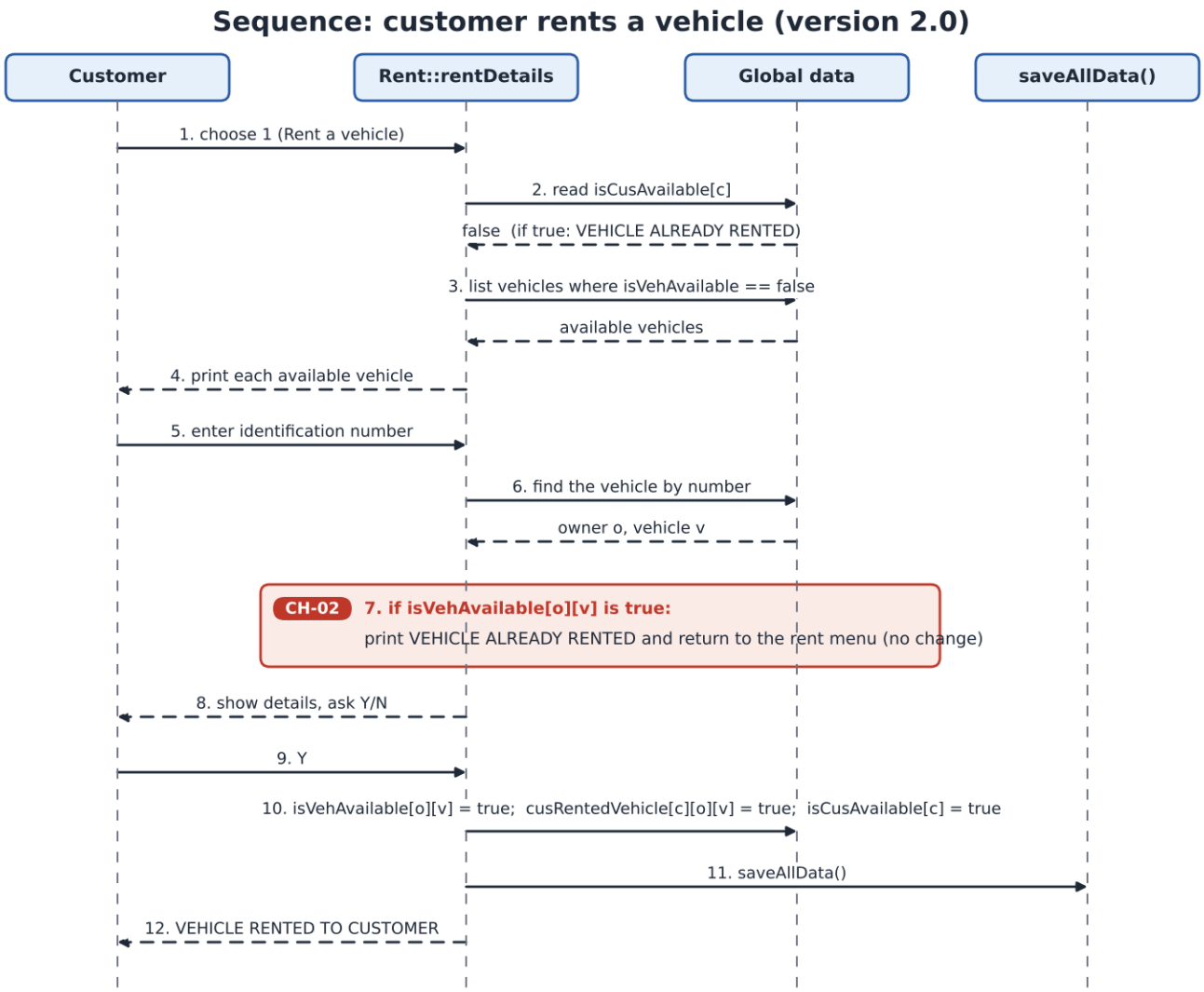
6.1 Class diagram: code structure

Menu functions create the four classes on demand; every class reads and writes the shared global arrays, and `saveAllData()` and `loadAllData()` move them to and from the text files. The red tags mark where CH-01 to CH-03 landed.



6.2 Sequence diagram: rent a vehicle

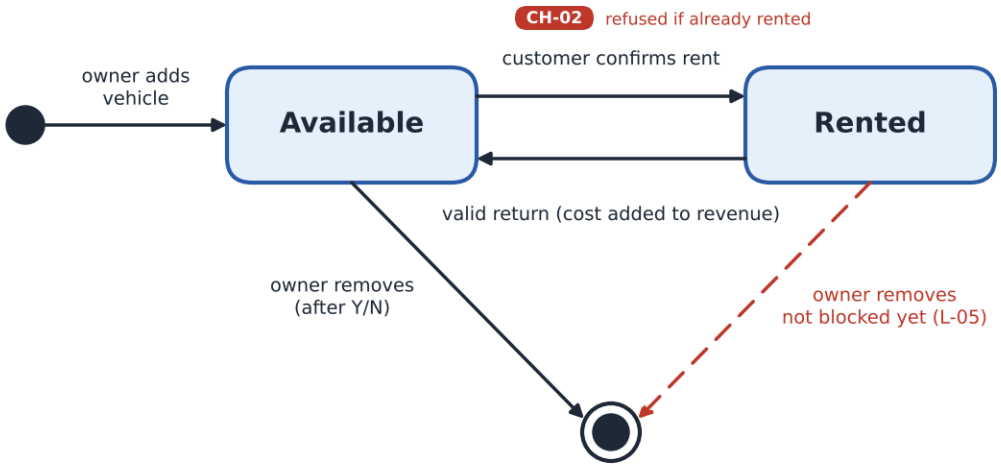
Step 7 is the CH-02 guard: a vehicle whose `isVehAvailable` flag is already true is refused before any data changes.



6.3 State diagram: vehicle

A vehicle moves between Available and Rented only through rent and return. The dashed red arrow is a gap, not a design: Remove does not check the rented flag yet (L-05).

State diagram: vehicle (version 2.0)



A rented vehicle should only leave through a valid return; today the Remove option does not check the rented flag.

7. Verification

The three changes were checked by one scripted console session on 31 Oct 2026: CH-02 and CH-03 passed, CH-01 passed on View Rented and failed on Return Vehicle.

The file builds with `g++ -std=c++11 -Wall -Wextra` and no warnings. Input was piped to the program in one run with two customers (Ali, Bilal), one owner and two vehicles (`ABC-1` , `XYZ-2`).

7.1 Test results

Test	Expected	Result
Ali rents `ABC-1` , then Bilal enters `ABC-1`	Refused	Pass: VEHICLE ALREADY RENTED (CH-02)
Bilal opens View Rented while Ali holds a vehicle	Bilal sees none of Ali's	Pass: NO VEHICLES RENTED (CH-01)
Bilal holds `XYZ-2` and opens Return Vehicle	Only Bilal's vehicle	**Fail:** Ali's `ABC-1` is offered first (CH-01 still open)
Type `abc` at a menu prompt	One error message, menu redisplayed	Pass (CH-03)
Start a return, then answer n	Vehicle stays rented	Pass

Test	Expected	Result
Choose Exit from the main menu	Program ends	**Fail:** earlier Return prompts reappear after PROGRAM ENDED (L-01)
Input ends while a menu is waiting	Data saved, program ends	**Fail:** crash, exit code 139 (L-02); data was already saved
Register a customer after an owner has added vehicles	Owner's vehicle count unchanged	**Fail:** the owner's 2 vehicles disappeared (line 165, defect 3)

7.2 Status of the 15 defects from version 1.0

Three of the 15 defects are fixed or partly fixed by CH-01 to CH-03, one is not present in this file, and 11 remain open.

#	Defect in v1.0	Status in v2.0
1	Menus call each other recursively; Exit unwinds old menus	Open (seen in the run)
2	Letters at a number prompt cause an endless loop	Partly fixed: menu prompts (CH-03)
3	Registering a customer erases an owner's vehicle count	Open (seen in the run)
4	Duplicate vehicle-number check compares the wrong arrays	Open
5	Maximum-vehicles message opens the customer portal	Open
6	A rented vehicle could be rented again	Fixed for normal use (CH-02)
7	Rented-vehicle screens show other customers' rentals	Partly fixed: View Rented (CH-01); Return open
8	Admin rented list printed NOT RENTED	Not present: the list prints RENTED
9	Search prints NOT FOUND after the menu returns	Open

#	Defect in v1.0	Status in v2.0
10	Removing an unknown number uses index -1	Open
11	A rented vehicle can be removed; IDs repeat after a removal	Open
12	Invalid choice or Exit jumps to the wrong menu	Open
13	`float` values, negatives and `|` in text accepted	Open
14	Two customers with the same name cannot both register	Open
15	ALL DATA SAVED after every change; misleading flag names	Open

8. Known limitations and next steps

The most urgent gaps are L-03 and L-04, because each lets one user's data affect another's, followed by L-01 and L-02, which cause the crashes seen in the run.

ID	Limitation	Why it matters	Suggested next step
L-03	Return Vehicle loops over every customer (line 797); it and View Rented bound the owner loop by `totalVehicles` (lines 760, 793), not `totalOwners`	A customer is offered, and could return, another customer's vehicle	Scan only `cusRentedVehicle[foundIndex][b][c]`, with `b` up to `totalOwners`
L-04	Registering a customer sets `vehPerOwn[totalCustomers]=0` (line 165)	An owner with the same index loses all vehicles from the count	Delete that line

ID	Limitation	Why it matters	Suggested next step
L-01	Menus call each other recursively	Exit unwinds old menus; earlier prompts reappear after PROGRAM ENDED	Turn each menu into a loop that returns to its caller
L-02	Mileage, rate, return-mileage and Y/N prompts are unguarded, and end of input is not handled	Bad input breaks `cin`; at end of input the menu guard recurses until the program crashes	Read whole lines and parse them; save and exit when `cin.eof()`
L-05	A rented vehicle can be removed by its owner	A rental is left pointing at nothing	Refuse removal while rented; renumber IDs and rental flags after a removal
L-06	Passwords are stored as plain text and the admin login is fixed in the source	Anyone who can open the data files can read every password	Store salted hashes; read admin credentials from a file
L-07	Arrays are fixed at 10 customers, 10 owners and 3 vehicles per owner	The system cannot grow	Replace arrays with structs in `std::vector`, keeping the same menus
L-08	One rental at a time, no booking dates, only a running revenue total	No reservations, receipts or per-owner earnings	Add start and end dates and append each rental to a history file
L-09	Tests are manual, scripted sessions	Regressions can slip in	Add an automated script that pipes input and compares output